

Parella de Punts Més Propera i Més Llunyana

Pràctica 3

Serafí Nebot Ginard

Universitat de les Illes Balears (UIB)

Grau en Enginyeria Informàtica

Algorismes Avançats, Curs 2025–2026

Resum—Aquesta memòria descriu el disseny i la implementació d'una aplicació Java que resol el problema de la parella de punts més propera (*closest pair*) i més llunyana (*farthest pair*) sobre un conjunt de punts en el pla. S'implementen quatre algorismes: força bruta $O(n^2)$, divideix i venceràs amb franja ordenada $O(n \log^2 n)$, una variant amb *buckets*, i la parella més llunyana mitjançant *convex hull* i *rotating calipers* $O(n \log n)$. L'aplicació separa la lògica, la interfície gràfica i el control en tres capes independents, ofereix una interfície gràfica interactiva amb visualització dels punts i resultats, i incorpora una eina de benchmark amb gràfiques i predicció de temps. Es presenta una anàlisi experimental rigorosa que confirma les complexitats teòriques i revela que l'optimització per *buckets* no aporta guany pràctic per al problema de la parella més propera.

Termes d'índex—parella més propera, parella més llunyana, divideix i venceràs, força bruta, convex hull, rotating calipers, Model-Vista-Controlador, benchmark, Java Swing

1. Introducció

El problema de la parella de punts més propera (*closest pair problem*) és un problema fonamental de la geometria computacional: donat un conjunt de n punts en el pla, trobar la parella de punts amb la distància euclidiana mínima. La solució per força bruta examina totes les $\binom{n}{2}$ parelles amb cost $O(n^2)$, però l'algorisme de divideix i venceràs redueix el cost a quasi lineal; en aquesta implementació la franja s'ordena a cada nivell, donant $O(n \log^2 n)$.

El problema complementari, la parella més llunyana (*farthest pair*), té la mateixa complexitat òptima $O(n \log n)$ però utilitza una estratègia diferent: primer es construeix el polígon convex que envolta tots els punts del núvol (*convex hull*) i després s'aplica la tècnica dels *rotating calipers*.

L'aplicació implementada permet generar núvols de punts amb quatre distribucions (uniforme, gaussiana, exponencial, agrupada), modificar paràmetres d'aquestes, executar els quatre algorismes, visualitzar els resultats gràficament i realitzar benchmarks sistemàtics amb predicció de temps.

2. Descripció de l'Aplicació

L'aplicació és una eina interactiva d'escriptori (Java Swing) que permet generar núvols de punts, trobar la parella més propera i més llunyana, i comparar el rendiment dels algorismes. La interfície es divideix en tres zones: un panell de controls a la part superior, la visualització de punts al centre i un panell de resultats a la dreta.

2.1. Generació de Punts

L'usuari configura el nombre de punts n amb un *spinner* i selecciona una de les quatre distribucions disponibles. Cada distribució té paràmetres configurables que es mostren dinàmicament:

- **Uniforme:** punts distribuïts uniformement dins del rang $[\min, \max]$ en ambdós eixos.
- **Gaussiana:** paràmetres de mitjana (fracció del rang), desviació estàndard σ (fracció del rang) i multiplicadors independents per a cada eix, permetent distribucions el·líptiques.
- **Exponencial:** factors λ_x i λ_y que controlen la densitat de punts cap a l'origen.
- **Agrupada (Clustered):** nombre de clusters (mínim 3), amb centres generats uniformement i punts distribuïts amb una gaussiana al voltant de cada centre.

2.2. Visualització del Núvol de Punts

El panell central mostra el núvol de punts generat. Cada punt es representa com un cercle sòlid de 4 px en gris fosc. El domini de coordenades $[0, 1000]^2$ es transforma linealment a les coordenades de pantalla amb un marge de 30 px a cada costat.

Quan s'executen els algorismes, les parelles resultants es ressalten sobre el núvol: la parella més propera en vermell i la més llunyana en blau. Cada ressaltat consisteix en una línia que connecta els dos punts i cercles de 8 px als extrems per destacar-los del núvol.

2.3. Execució i Resultats

El botó *Executar* llança els quatre algorismes en un fil de fons per no bloquejar la interfície. Durant l'execució, els controls es deshabiliten i el cursor canvia a mode d'espera. El botó *Aturar* permet cancel·lar l'execució cooperativament.

El panell de resultats a la dreta de la finestra mostra, per a cada algorisme, les coordenades i distància de la parella trobada i el temps d'execució mesurat. Mentre no s'ha executat cap algorisme, tots els valors mostren «—».

Cada algorisme s'executa amb el protocol de la Secció ??: escalfament JIT global al principi de la sèrie i, després, 1 execució mesurada per cas (algorisme, n).

2.4. Finestra de Benchmark

El botó *Benchmark* obre una finestra independent no modal per executar benchmarks sistemàtics. La finestra s'organitza en quatre zones:

- **Configuració (superior):** camps per introduir els valors de n separats per comes, un selector de distribució i els paràmetres de la distribució triada.
- **Gràfica (centre):** mostra les quatre corbes de temps en funció de n . Permet activar escala logarítmica i amagar series individuals. Superposa corbes ajustades (línia discontinua) un cop finalitzat el benchmark.
- **Predicció (dreta):** un cop finalitzat el benchmark, ajusta les constants multiplicatives dels models teòrics i permet introduir un valor de n arbitrari per predir el temps d'execució de cada algorisme.
- **Taula de dades (inferior):** mostra els temps mesurats per a cada n i algorisme en format tabular.

3. Disseny i Implementació

3.1. Estructura de l'Aplicació

L'aplicació es divideix en tres capes amb responsabilitats ben separades, que es comuniquen per events:

- 1) **Model:** la classe `Model` encapsula el núvol de punts, els resultats de cada algorisme (parella i temps), i els resultats del benchmark. Només emmagatzema dades; els algorismes viuen en classes separades dins la mateixa capa.
- 2) **Vista:** conjunt de components Swing que presenten els controls, la visualització i la gràfica. No contenen lògica de negoci ni dades del problema.
- 3) **Controlador:** implementa la interfície `ViewListener` per rebre els events de la vista, gestiona el cicle de vida dels fils de fons i actualitza el model i la vista.

La interfície `ViewListener` defineix quatre events: `onGenerate`, `onRun`, `onStop` i `onOpenBenchmark`. La vista emet events sense conèixer qui els processa, i

el controlador implementa la interfície sense conèixer els components gràfics interns de la vista.

Degut a que la vista actualitza automàticament el núvol de punts cada vegada que l'usuari modifica alguna dada de configuració, existeix un temporitzador *debounce* de 300 ms evita emetre l'event `onGenerate` a cada tecleig: la vista només el llança quan l'usuari ha deixat d'interactuar durant 300 ms.

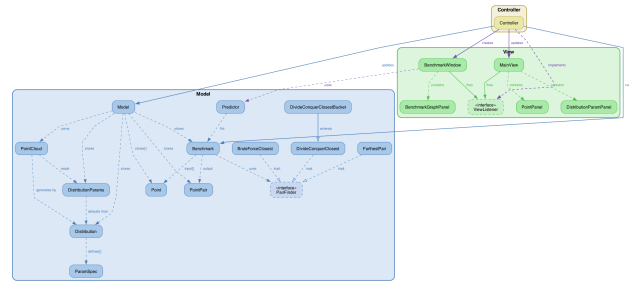


Figura 1. Diagrama de classes de l'aplicació amb les relacions entre les tres capes.

3.2. Capa del Model

La capa del model conté les classes de dades i els algorismes. No importa cap classe de la vista ni del controlador.

3.2.1. Model. La classe `Model` és el contenidor central d'estat de l'aplicació. Emmagatzema el núvol de punts actual (`Point[]`), la distribució i els paràmetres usats per generar-lo (`Distribution`, `ParamSpec/DistributionParams`), els resultats de cada algorisme (`PointPair` i temps en ms) i els resultats del darrer benchmark (`BenchmarkEntry[]`). No conté cap lògica d'algorisme: és un magatzem de dades pur que el controlador escriu i la vista llegeix a través dels mètodes d'accés.

3.2.2. Point i PointPair. `Point` és un registre immutable amb dos camps `double` (x , y). Ofereix `distanceTo()` que calcula la distància euclidiana entre dos punts.

`PointPair` encapsula una parella de punts i calcula la distància de forma *lazy*: el camp `distance` s'inicialitza a -1 com a sentinella i es calcula només la primera vegada que es demana. Gràcies a això, els algorismes que construeixen molts `PointPair` intermedis només paguen el cost de l'arrel quadrada quan realment es necessita la distància.

3.2.3. Distribution, PointCloud, DistributionParams i ParamSpec. L'enumeració `Distribution` defineix les distribucions disponibles (uniforme, gaussiana, exponencial i agrupada) i el seu nom de presentació. `PointCloud` implementa aquestes distribucions usant `java.util.Random`.

Els paràmetres de cada distribució es defineixen com a arrays de `ParamSpec` (registre amb nom i valor per defecte), i els valors actuals s'encapsulen en

DistributionParams (registre immutable amb còpia defensiva al constructor). Aquest disseny permet que la vista construeixi els camps de configuració automàticament a partir de ParamSpec[], sense conèixer quines distribucions existeixen ni quants paràmetres té cadascuna. D'aquesta forma, si en el futur s'afageix, elimina o es modifica qualche distribució o paràmetre no fa falta modificar la vista, aquesta mostrarà totes les opcions automàticament.

3.2.4. Benchmark i Predictor. La classe Benchmark centralitza el protocol de mesura i l'execució paral·lela. El protocol és:

- 1) **Escalfament JIT global:** s'executen tots els algorismes 10 vegades amb $n = 500$ abans d'iniciar una sèrie de benchmark (finestra/CLI) per forçar la compilació JIT i així descartar possibles interferències amb el temps d'execució mesurat.
- 2) **Mesura:** una execució mesurada amb `System.nanoTime()`, convertida a mil·lisegons.

Per a l'execució individual des de la interfície principal s'aplica directament la fase de mesura.

Per al benchmark en sèrie, `Benchmark.runSeries` genera els arrays de punts una sola vegada al fil principal i després distribueix les tasques amb un `ExecutorService` de granularitat fina: una tasca per parella (algorisme, n), de manera que els quatre algorismes per al n més gran s'executen en paral·lel en nuclis separats.

La classe Predictor ajusta una constant multiplicativa a per a cada algorisme amb mínims quadrats ponderats (sense ordenada a l'origen):

$$a = \frac{\sum_i t_i \cdot f(n_i)}{\sum_i f(n_i)^2} \quad (1)$$

on n_i i t_i són la quantitat de punts del núvol i el temps d'execució mesurat per a l'execució i del benchmark. La funció $f(n_i)$ equival al temps teòric per a cada un dels algorismes. La ponderació dona més pes a les mesures amb n gran, on el comportament asimptòtic domina la sobrecàrrega de la JVM.

3.2.5. Interfície Comuna dels Algorismes. Tots els algorismes implementen la mateixa interfície `PairFinder`:

```
1 public interface PairFinder {
2     PointPair find(Point[] points);
3 }
```

Listing 1. Interfície comuna per a tots els algorismes.

Això permet al benchmark i al controlador cridar qual-sevol algorisme sense saber quin és concretament.

Pel que fa a les dues variants de D&C, `DivideConquerClosest` implementa l'algorisme complet (ordenar per X , recursió, construir franja, processar franja). `DivideConquerClosestBucket` en reutilitza tot menys l'últim pas: sobreesciu només el mètode `processStrip()` per usar `buckets` en lloc d'ordenar per Y . D'aquesta manera no es repeteix cap codi: tot el que és comú entre les dues variants viu en un sol lloc.

3.2.6. Algorisme de Força Bruta. L'algorisme de força bruta (`BruteForceClosest`) examina totes les $\binom{n}{2} = \frac{n(n-1)}{2}$ parelles de punts i retorna la de distància mínima.

```
1 for (int i = 0; i < n - 1; i++) {
2     for (int j = i + 1; j < n; j++) {
3         double dist = points[i].distanceTo(points[j]);
4         if (dist < bestDist) {
5             bestDist = dist;
6             bestP1 = points[i];
7             bestP2 = points[j];
8         }
9     }
10 }
```

Listing 2. Nucli de l'algorisme de força bruta.

Cost temporal. L'algorisme avalua totes les $\binom{n}{2} = \frac{n(n-1)}{2}$ parelles, per tant $T(n) \in O(n^2)$.

Cost espacial. L'espai addicional és constant, ja que només es mantenen variables auxiliars i la millor parella: $O(1)$.

Notes pràctiques. La implementació comprova `Thread.interrupted()` a cada iteració del bucle extern per permetre la cancel·lació cooperativa des de la interfície.

3.2.7. Algorisme de Divideix i Venceràs. L'algorisme de divideix i venceràs (`DivideConquerClosest`) segueix les següents passes:

- 1) Ordenar els punts per coordenada x una sola vegada ($O(n \log n)$).
- 2) Recursivament trobar la parella més propera a la meitat esquerra i la meitat dreta.
- 3) Construir la franja central de punts amb $|x - x_{\text{mid}}| < d_{\text{min}}$.
- 4) Processar la franja: ordenar per y i per a cada punt, comparar amb els 7 veïns següents (argument de l'empaquetament).

El cas base s'activa quan el subproblema té 3 o menys punts, resolent-se per força bruta amb cost $O(1)$.

Argument d'empaquetament (per què 7 veïns). Després de resoldre les dues meitats, d_{min} és la millor distància dins de cada costat. En la franja central (amplada $2d_{\text{min}}$), per a un punt fixat només poden ser candidats els punts amb diferència en y menor que d_{min} . Això defineix un rectangle de mida $2d_{\text{min}} \times d_{\text{min}}$. Si el subdividim en cel·les de costat $d_{\text{min}}/2$, obtenim 8 cel·les; no hi pot haver dos punts a la mateixa cel·la perquè estarien a distància menor que d_{min} (contradicció). Per tant, hi ha com a màxim 8 punts rellevants en aquesta zona i, ordenant per y , n'hi ha prou de comparar cada punt amb els 7 següents.

Cost temporal.

$$T(n) = 2T(n/2) + f(n) \quad (2)$$

on $f(n) \in O(n \log n)$ prové del processament de la franja a cada nivell: ordenació per y ($O(k \log k)$ amb $k \leq n$) i comparacions acotades entre veïns a la franja (cost lineal sobre k , amb un màxim constant de comparacions per punt).

Podem resoldre la recurrència amb el *Teorema Mestre*:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

on a és el nombre de subproblemes, n/b la mida de cada subproblema, i $f(n)$ el cost de combinar resultats.

En el nostre cas:

- $a = 2$ (dues meitats),
- $b = 2$ (cada meitat té mida $n/2$),
- $f(n) = n \log n$ (cost de processar la franja).

Com que el treball recursiu pur és d'ordre n (ja que $n^{\log_b a} = n^{\log_2 2} = n$) i aquí la combinació és un factor $\log n$ més gran ($n \log n$), la forma generalitzada del Teorema Mestre dona:

$$T(n) \in O(n \log^2 n)$$

Cost espacial. L'espai és $O(n)$ (buffer de franja preal·locat + pila recursiva).

Notes pràctiques. Les optimitzacions següents no canvien l'ordre asimptòtic, però redueixen constants i milloren el rendiment real de l'algorisme.

Optimització 1 – Cerca binària per als límits de la franja.

En lloc d'un escaneig lineal $O(k)$ per trobar els punts dins de la franja, s'utilitza cerca binària $O(\log k)$ amb els mètodes `lowerBound` i `upperBound`, aprofitant que els punts ja estan ordenats per x . Això no millora la complexitat asimptòtica (el pas dominant segueix sent l'ordenació de la franja) però redueix el treball constant.

Optimització 2 – Buffer preal·locat per a la franja.

A cada nivell de recursió, l'algorisme necessita copiar els punts de la franja central a un buffer temporal per ordenar-los per y . La implementació ingènua crearia un nou array a cada crida recursiva, generant $O(\log n)$ al·locacions (una per nivell).

Per eliminar la pressió sobre el *garbage collector* i millorar la localitat de cache, s'al·loca un únic array `Point[n]` al mètode `find()` i es reutilitza a tots els nivells de recursió. El buffer és segur perquè cada nivell de recursió l'omple amb `System.arraycopy` i només accedeix als índexos $[0, k)$ on k és el nombre de punts de la franja actual. Els nivells de recursió no es trepitgen perquè l'execució és seqüencial (un sol fil): un nivell pare copia la seva franja, fa la crida al nivell fill i queda aturat fins que el fill retorna. Si el fill sobreescriu el buffer, no passa res: el pare ja no necessita les dades antigues.

3.2.8. Variant amb Buckets. La variant amb *buckets* (`DivideConquerClosestBucket`) intenta millorar el pas de processament de la franja. En lloc d'ordenar la franja per y amb cost $O(k \log k)$, distribueix els punts en cubetes (*buckets*) d'alçada $d_{\min}$ i comprova només la cubeta actual i la immediatament superior. Per l'argument de l'empaquetament explicat abans, cada cubeta conté com a màxim 8 punts, de manera que el cost per punt és $O(1)$ i el cost total de la franja és $O(k)$.

Cost temporal. La millora damunt del DC la recurrència es redueix de $T(n) = 2T(n/2) + k \log k$ a $T(n) = 2T(n/2) + k$. Com que $k \leq n$, llavors $T(n) \in O(n \log n)$.

Cost espacial. El consum addicional és lineal en el pitjor cas ($O(n)$ per al buffer i estructures de franja), però amb constants més altes que D&C per la gestió de cubetes.

Notes pràctiques. Cost de Cubetes Disperses. L'optimització per *buckets* requereix crear un array de cubetes indexat pel rang de coordenada y . El nombre de cubetes és:

$$\text{numBuckets} = \left\lceil \frac{y_{\max}}{d_{\min}} \right\rceil - \left\lceil \frac{y_{\min}}{d_{\min}} \right\rceil + 1 \quad (3)$$

Quan $\text{numBuckets} \gg k$ (molt més cubetes que punts), l'array és molt dispers i el cost d'al·locació passa a dominar. En aquesta implementació no s'utilitza cap guarda: l'objectiu és observar el cost real d'aquesta estratègia en la pràctica.

Limitació Pràctica Fonamental. Durant l'anàlisi experimental s'ha observat que, per al problema de la parella més propera, $d_{\min}$ decreix ràpidament i el nombre de cubetes esdevé enorme. Per a un núvol de punts amb rang R en l'eix y , el nombre de cubetes és $\text{numBuckets} \approx R/d_{\min}$, mentre que el nombre de punts a la franja és $k \leq n$. Com que $d_{\min} \ll R$, es té $\text{numBuckets} \gg k$.

El resultat és que la variant amb *buckets* **no aporta cap guany pràctic** i sovint és més lenta que D&C: la creació d'un array dispers gran introdueix sobrecost d'al·locació i pressió sobre el *garbage collector*. Aquesta limitació és estructural per a distribucions contínues habituals, i explica la degradació empírica.

L'optimització per *buckets* seria útil en contextos on $d_{\min}$ és comparable al rang de la franja, com ara problemes on la distància mínima és gran (p. ex. conjunts de punts en graelles regulars) o problemes de *range searching*.

3.2.9. Parella Més Llunyana. L'algorisme de la parella més llunyana (`FarthestPair`) combina dues fases: (1) càlcul de l'envolupant convexa amb la cadena monòtona d'Andrew [?], i (2) cerca del parell antipodal màxim amb *rotating calipers* [?].

Cost temporal. Com que el *convex hull* costa $O(n \log n)$ i els *rotating calipers* costen $O(h)$, el cost total és $T(n) \in O(n \log n)$.

Cost espacial. L'espai addicional és lineal per emmagatzemar l'envolupant convexa i estructures auxiliars: $O(n)$.

Notes pràctiques. La fase de *rotating calipers* és molt lleugera; en la pràctica el temps queda dominat pel càlcul de l'envolupant convexa.

3.3. Capa de la Vista

La capa de la vista consta de cinc components Swing principals, cadascun amb una responsabilitat concreta.

3.3.1. MainView. La finestra principal usa `BorderLayout`: el panell de controls a dalt (NORTH), el `PointPanel` al centre i el panell de resultats a la dreta (EAST). El panell de controls combina dos subpanells amb `FlowLayout`: un a l'esquerra amb el

JSpinner de n , el JComboBox de distribució i el DistributionParamPanel, i un a la dreta amb els botons *Executar*, *Aturar* i *Benchmark*.

La comunicació cap al controlador es fa exclusivament a través de `ViewListener`: el botó *Executar* crida `onRun`, *Aturar* crida `onStop`, i *Benchmark* crida `onOpenBenchmark`. La vista no coneix el controlador; simplement invoca l'event sobre el listener registrat.

El panell de resultats organitza vuit etiquetes en vertical amb `BoxLayout`: quatre per a les parelles trobades (coordenades i distància) i quatre per als temps d'execució. Quan s'ha generat un núvol nou però encara no s'han executat algorismes, les etiquetes mostren «—» i el botó *Executar* s'habilita per indicar que les dades estan llestes.

3.3.2. PointPanel. El renderitzat es fa directament a `paintComponent` amb tres capes seqüencials: primer tots els punts del núvol en gris, després la parella llunyana en blau (si disponible) i finalment la parella propera en vermell (si disponible). Renderitzar la parella propera en darrer lloc garanteix que sempre queda visible sobre la llunyana en cas que els extrems coincideixin.

La transformació de domini a pantalla és:

$$x_{px} = \text{MARGIN} + \frac{x - x_{\min}}{x_{\max} - x_{\min}} \cdot w \quad (4)$$

$$y_{px} = \text{MARGIN} + h - \frac{y - y_{\min}}{y_{\max} - y_{\min}} \cdot h \quad (5)$$

on w i h són les dimensions útils del panell (descomptant els marges). La subtracció de h a la fórmula de y inverteix l'eix perquè la pantalla té l'origen a dalt i el domini matemàtic a baix.

3.3.3. DistributionParamPanel. Aquest panell genera els seus camps de text dinàmicament en el moment en què es canvia la distribució seleccionada. Recorre l'array `ParamSpec[]` de la distribució i crea un parell `JLabel` + `JTextField` per a cada paràmetre, inicialitzant-lo amb el valor per defecte de l'especificació. Cada camp té un `DocumentListener` que, quan el contingut canvia, reinicia el temporitzador de *debounce* de `MainView`, el qual al seu torn emet l'event `onGenerate` al `ViewListener` un cop transcorreguts els 300 ms. Si l'usuari introdueix un valor no numèric, el mètode `read()` retorna silenciosament el valor per defecte sense mostrar cap error.

Gràcies a aquest disseny, afegir una nova distribució o modificar els seus paràmetres no requereix cap canvi a la vista.

3.3.4. BenchmarkWindow. La finestra de benchmark és un `JDialog` no modal. Utilitza `BorderLayout` amb la configuració a dalt, la gràfica al centre, un panell d'opcions i predicció a la dreta, i la taula de dades a baix.

De manera idèntica a `MainView`, `BenchmarkWindow` es comunica amb el controlador a través de `ViewListener`. El mètode `onRunBenchmark()` forma part de la mateixa interfície que `onRun`, `onStop`, etc., de manera que el controlador

és l'únic punt d'implementació i la finestra no coneix en cap moment el seu tipus concret.

L'execució del benchmark es delega a un `SwingWorker`: el fil EDT resta responent durant l'execució. Quan el `SwingWorker` acaba, el mètode `done()` s'executa a l'EDT, actualitza la gràfica, la taula i ajusta les constants del predictor, i habilita el botó de predicció.

3.3.5. BenchmarkGraphPanel. La gràfica es dibuixa completament a `paintComponent`, sense cap llibreria externa. El mètode `generateRoundTicks()` calcula etiquetes d'eixos arrodonides a potències de $10 \times \{1, 2, 5\}$ per obtenir graelles netes independentment del rang de valors.

En mode logarítmic, l'eix y es transforma amb $\log_{10}$ i les divisions secundàries es col·loquen als múltiples $\times 2$ i $\times 5$ de cada potència de 10. En mode lineal, les divisions es calculen amb `generateRoundTicks`.

Les corbes ajustades es dibuixen amb una línia discontinua (`BasicStroke` amb array de *dash*) en un to lleugerament més fosc que la sèrie de dades corresponent, mostrant el model $a \cdot f(n)$ mostrejat a un píxel per columna.

3.4. Capa del Controlador

La capa del controlador consisteix en una sola classe, `Controller`, que implementa `ViewListener` i conté tota la lògica d'orquestració de l'aplicació.

3.4.1. Gestió dels Events de la Vista. Cada mètode de `ViewListener` correspon a una acció de l'usuari:

- **onGenerate:** crida `model.generatePoints()` amb els paràmetres rebuts, i seguidament `view.showPoints()` per actualitzar la visualització i `view.clearResults()` per esborrar els resultats anteriors. Finalment habilita el botó *Executar*.
- **onRun:** crea un `SwingWorker` que executa els quatre algorismes sobre el núvol actual al fil de fons, desa els resultats al model amb `model.setResults()`, i en acabar crida `view.updateResults()` a l'EDT. Durant l'execució, crida `view.setRunning(true)` per deshabilitar els controls i activar el cursor d'espera.
- **onStop:** crida `worker.cancel(true)`, que interromp el fil de fons. La cancel·lació és cooperativa: `BruteForceClosest` comprova `Thread.isInterrupted()` a cada iteració del bucle extern i llança una excepció si detecta la interrupció.
- **onOpenBenchmark:** crea la `BenchmarkWindow` (si no existeix ja) passant-se a si mateix com a `ViewListener`, i la fa visible.

3.5. CLI de Benchmark

A més de la interfície gràfica, s'ofereix una eina de línia de comandes (`BenchmarkCLI`) que permet execu-

tar benchmarks amb paràmetres configurables i sortida en format taula o CSV:

```

1 # Tots els algorismes, distribucio uniforme
2 java -cp target/classes \
3   com.serafinebot.p3.BenchmarkCLI
4
5 # D&C vs Bucket, gaussiana, CSV
6 java -cp target/classes \
7   com.serafinebot.p3.BenchmarkCLI \
8   -a dc,bucket -d GAUSSIAN --csv
9
10 # 3 execucions promitjades
11 java -cp target/classes \
12   com.serafinebot.p3.BenchmarkCLI -r 3

```

Listing 3. Exemples d'ús del CLI.

4. Anàlisi Experimental

Totes les mesures s'han realitzat amb el CLI de benchmark, amb 3 execucions independents mitjanes, en una màquina amb processador Apple M1 (8 nuclis), 16 GB RAM i JDK 17. Els punts s'han generat en el rang $[0, 1000]$, i s'han provat les distribucions uniforme, gaussiana ($\mu = 0.5$, $\sigma = 1/6$), exponencial ($\lambda_x = \lambda_y = 5$) i agrupada (5 clusters). L'execució del benchmark distribueix les tasques entre nuclis; això pot afegir soroll en temps molt petits però no afecta les tendències globals. Com a cas addicional s'ha provat una gaussiana estirada ($\sigma = 0.05$, multiplicadors $\times 4$ i $\times 0.25$) per explorar un núvol anisotròpic. Els temps petits (< 0.1 ms) presenten variabilitat elevada; s'analitzen sobretot tendències i valors per a n grans.

4.1. Distribució Uniforme: Comparació Completa

La Taula ?? recull els temps mitjans per a la distribució uniforme i la Figura ?? mostra el creixement en escala logarítmica.

Taula 1. TEMPS D'EXECUCIÓ (MS) PER A DISTRIBUCIÓ UNIFORME.

n	Bruta	D&C	D&C Bkt.	Llunyana
100	0.006	0.088	0.331	0.069
500	0.148	1.855	1.044	0.791
1000	0.576	1.805	1.834	0.540
2000	3.020	3.983	3.192	1.733
5000	19.964	5.379	5.790	5.763
10000	64.357	12.072	12.096	2.981
20000	233.536	10.711	13.280	7.388
50000	1486.131	30.598	34.526	18.586

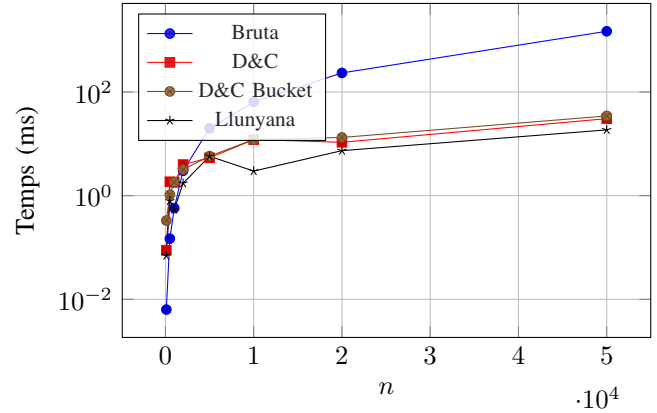


Figura 2. Distribució uniforme: temps vs n (escala logarítmica).

Els resultats mostren el patró esperat: força bruta domina per a n petits per la seva baixa constant, però queda ràpidament superada per D&C i la parella llunyana. Per $n = 50\,000$, D&C és $\approx 49\times$ més ràpid que força bruta, mentre que la parella llunyana és la més ràpida dels algorismes quasi-lineals. La variant amb *buckets* és sistemàticament més lenta que D&C en la distribució uniforme ($\sim 13\%$ a $n = 50\,000$).

4.2. Comparació entre Distribucions

La Taula ?? resumeix els temps a $n = 50\,000$ per a totes les distribucions i el cas gaussià estirat. La Figura ?? compara explícitament D&C i la variant amb *buckets*.

Taula 2. TEMPS (MS) PER ALGORISME I DISTRIBUCIÓ A $n = 50\,000$.

Distribució	Bruta	D&C	D&C Bkt.	Llunyana
Uniforme	1486.131	30.598	34.526	18.586
Gaussiana	1466.164	30.296	38.160	18.004
Exponencial	1508.508	27.184	40.977	16.277
Agrupada (5)	1493.985	29.404	36.879	17.480
Gaussiana estirada	1470.742	25.154	23.590	17.275

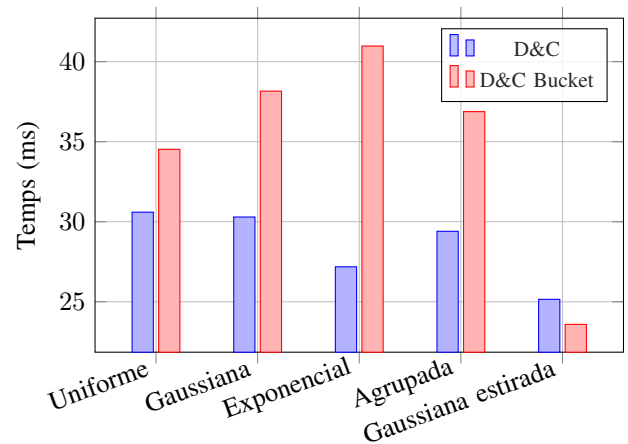


Figura 3. D&C vs D&C Bucket per distribució a $n = 50\,000$.

Els temps varien poc entre distribucions: la geometria afecta sobretot constants menors, però no canvia l'ordre de magnitud. D&C es manté estable, mentre que la variant amb *buckets* és més variable: en distribucions uniforme, gaussiana i exponencial és clarament més lenta, però en la gaussiana estirada esdevé lleugerament més ràpida perquè el rang efectiu en y és més petit. La parella llunyana es manté estable perquè el cost depèn del *convex hull*, que és relativament petit en totes les distribucions provades.

4.3. Cas d'Estudi: Gaussiana Estirada

La distribució gaussiana estirada (ell·líptica) serveix per comprovar si una geometria anisotròpica altera el comportament. En aquest cas el rang efectiu en l'eix y és més petit i el nombre de cubetes baixa, de manera que la variant amb *buckets* pot superar D&C per a n grans (23.59 ms vs 25.15 ms a $n = 50\,000$). Tot i això, el comportament no és monòton per a tots els n , mostrant sensibilitat a la geometria i al soroll de mesura.

4.4. Problemes Pràctics de la Variant amb Buckets

La Figura ?? mostra el ràtio Bucket/D&C per a la distribució uniforme i la gaussiana estirada. En uniforme el ràtio és major que 1 per a n grans, evidenciant el sobrecost d'al·locar cubetes disperses. En la gaussiana estirada el ràtio pot baixar per sota d'1 quan el rang efectiu en y és reduït, però el patró és irregular. Els valors erràtics per a n petits reflecteixen l'overhead de construir les cubetes quan el temps absolut és molt baix i el soroll de mesura domina.

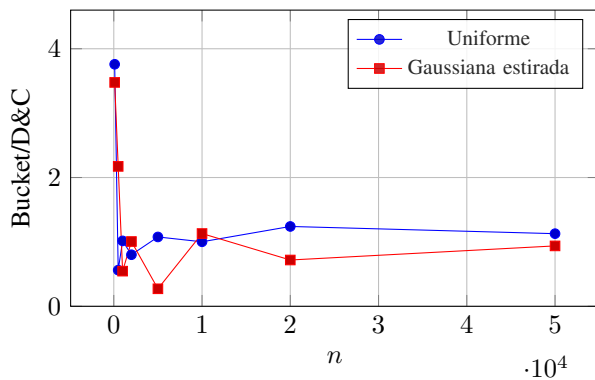


Figura 4. Ràtio Bucket/D&C per n .

Aquest comportament és coherent amb la propietat clau del problema: a mesura que la recursió avança, $d_{\min}$ es fa molt petit i el nombre de cubetes `numBuckets` esdevé molt major que k . Sense guarda, això implica al·locacions disperses massives i pressió sobre el GC. Per tant, la variant amb *buckets* és teòricament atractiva però pràcticament ineficaç en distribucions contínues estàndard, i només pot ser competitiva quan el rang efectiu en y és prou reduït.

5. Decisions de Disseny Notables

5.1. Herència vs Duplicació de Codi

La decisió de fer que `DivideConquerClosestBucket` hereti de `DivideConquerClosest` té un compromís: la crida al mètode sobreescrit `processStrip` afegeix un overhead mínim. En la pràctica, el cost és insignificant (< 1 ns per crida) comparat amb el cost de la franja (> 1 μ s fins i tot per a franges petites), i s'estalvien ≈ 125 línies de codi duplicat.

5.2. Càlcul de distància simplificat

S'ha optat per una API mínima de distància: `Point` només exposa `distanceTo()`, i `PointPair` calcula i memoritza aquesta distància en una sola passada. Aquest enfocament evita mantenir dos camins paral·lels (distància real i distància al quadrat), redueix superfície d'API i simplifica la lectura del codi sense impacte rellevant en el rendiment observat.

5.3. Buffer Preallocat vs Al·locació per Nivell

El buffer preallocat `Point[n]` per a la franja elimina $O(\log n)$ al·locacions per crida a `find()`, reduint la pressió sobre el GC i millorant la localitat de cache. El compromís és que el buffer té mida n (la mida màxima possible de la franja és n al nivell arrel) tot i que els nivells inferiors utilitzen fraccions cada vegada més petites.

6. Conclusió i Opinió Personal

En aquesta pràctica s'ha implementat una aplicació completa per al problema de la parella de punts més propera i més llunyana, amb una arquitectura MVC clara, visualització gràfica i una eina de benchmark específica. El disseny ha permès separar la generació de punts, l'execució dels algorismes i la presentació de resultats sense acoblaments innecessaris.

Els resultats experimentals confirmen la transició esperada entre força bruta i els algorismes quasi-lineals: la força bruta és competitiva per a n petits per la seva baixa constant, però D&C domina a partir de valors moderats i la parella llunyana és especialment eficient per a n grans. La variant amb *buckets* mostra un sobrecost important en distribucions contínues, degut a cubetes disperses i pressió sobre el GC, i només resulta competitiva en casos anisotròpics molt concrets.

Com a treball futur, seria interessant implementar la variant òptima $O(n \log n)$ amb franja preordenada, incorporar mesures de variància i seeding determinista per reforçar la reproductibilitat, i explorar distribucions o rangs més extrems per estudiar casos límit.

Referències

- [1] M. I. Shamos i D. Hoey, "Closest-point problems," *Proc. 16th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 151–162, 1975.
- [2] G. T. Toussaint, "Solving geometric problems with the rotating calipers," *Proc. IEEE MELECON*, pp. A10.02/1–4, 1983.
- [3] A. M. Andrew, "Another efficient algorithm for convex hulls in two dimensions," *Information Processing Letters*, vol. 9, no. 5, pp. 216–219, 1979.